Kinematic analysis of structural mechanics based on AI Agent

Leye Zhang[1], Hongjun Zhang[2,*]

(1. Nanjing Audit University Jinshen College, Nanjing, 210023, China;

2. Wanshi Antecedence Digital Intelligence Traffic Technology Co., Ltd, Nanjing, 210016, China)

Abstract: Kinematic analysis is a foundational task in structural mechanics for determining the geometric composition and stability of planar bar systems. This paper develops an agent system based on the LangChain framework for structural mechanics kinematic analysis. The system takes a large language model as its reasoning core and integrates two skills: one is the doubao-visualization skill package, which provides image understanding and rendering capabilities to recognize input structural example images and draw schematic diagrams annotated with analysis results; the other is the structural-kinematic-analysis professional skill package, built according to structural mechanics textbooks, which can parse nodes, bars, and supports, compute the degrees of freedom of the system, and invoke rules such as the binary-body, two-rigid-plate, and three-rigid-plate rules to perform kinematic analysis. By merely inputting a structural image, the agent can autonomously plan and invoke the required skills, output a verdict on whether the system is geometrically stable or unstable together with visualized results, and, thanks to its memory capability, support interactive follow-up questions such as "Is it still stable if a certain support is removed?" Tests on two typical examples of a geometrically stable system and a geometrically unstable system show that the agent produces correct results, verifying the feasibility of the proposed approach and offering a new pathway toward intelligent assistance for structural mechanics teaching and the preliminary assessment of engineering structures.

Keywords: AI Agent; large language model; skills; structural mechanics; kinematic analysis; geometrically stable system; geometrically unstable system

0 Introduction

A structure is the skeleton that supports loads in buildings. Typical examples include beam-column systems in buildings, dams in hydraulic engineering, and bridges on highways and railways. According to their geometric form, structures are classified into bar structures, plate-and-shell structures, and solid structures. Engineering structures are designed to support loads, and their geometric shape and position must remain stable. Therefore, kinematic analysis must be carried out before structural design. The kinematic analysis of planar bar structures is the most fundamental topic in structural mechanics [1,2].

In the past two years, AI Agent technology has made remarkable progress and has gradually penetrated various industries. Lingyan Zhang constructed an agent

Leye Zhang (2003—), Male, Students of Data Science and Big Data Technology in Class of 2024 (college-to-university transfer program).

*Corresponding author: Hongjun Zhang (1969–), Male, Class I Registered Structural Engineer. Research direction: road and bridge design, artificial intelligence. E-mail: 583304953@qq.com.

for Alzheimer's diagnosis and treatment based on renowned-physician knowledge using chain-of-thought, retrieval-augmented generation, and prompt engineering techniques [3]; Hanyue Mao applied multi-agent technology to the development of digital educational resources [4]; Rongrong Chen designed an LLM-driven agent to intelligentize novelty search [5]; Fan Ma developed an educational assessment agent oriented to multi-dimensional scenarios [6]. However, no research has yet applied AI Agent technology to the kinematic analysis of structural mechanics.

An agent is an intelligent software entity that perceives its environment, makes decisions, and executes actions to achieve specific goals. It is not merely a chatbot that answers questions, but an intelligent executor that can actually get things done. This paper constructs an agent using the LangChain framework, invoking a large language model and skills to perform kinematic analysis of planar bar systems (open source address of this article's dataset and source code :https://github.com/ zhangleye/Kinematic-Analysis-Agent).

1 Introduction to Structural Mechanics Kinematic Analysis and AI Agent

1.1 Kinematic analysis of structural mechanics

For convenience, only planar bar structures are discussed here; material deformation is not considered in the analysis.

1.Geometrically stable system: under arbitrary loads, the geometric shape of the structure remains unchanged, such as the truss in Fig. 1.

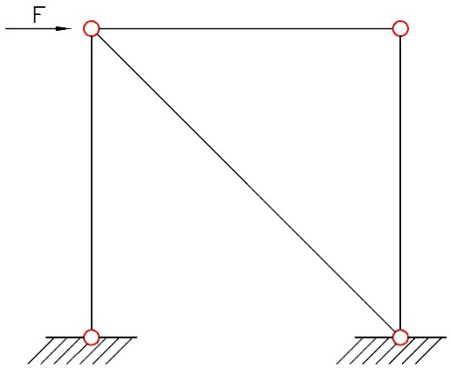


Fig. 1 Example of a geometrically stable system

2.Geometrically unstable system: under even very small loads, the geometric shape of the structure cannot remain unchanged, such as the truss in Fig. 2.

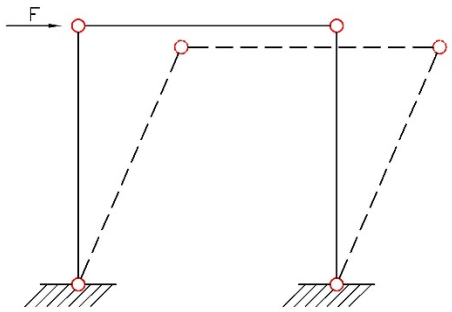


Fig. 2 Example of a geometrically unstable system

In Fig. 1 and Fig. 2, F denotes the external load, and the dashed line shows

the geometric shape at a certain moment during mechanical motion.

Engineering structures must generally be geometrically stable systems; geometrically unstable systems cannot be adopted, as they would be unable to maintain equilibrium under loads.

1.2 AI Agent

An agent is an AI system that autonomously determines part of its operational steps according to a goal and invokes tools to complete tasks[7]. Put more simply, an agent is an intelligent assistant that can get things done. AI Agent = LLM (brain) + Planning + Tool use + Memory.

Traditional software programs follow a fixed instruction flow: input → processing → output. An agent, by contrast, is more like an autonomous employee that can: (1) understand the task goal—comprehend what result is desired; (2) make plans—think about how to achieve the goal; (3) use tools—invoke various resources and APIs; (4) self-adjust—optimize strategies based on feedback; and (5) execute continuously—until the task is completed or an unsolvable problem is encountered.

Agent workflow: input → reasoning → tool invocation → execution → result return → continuous iteration.

2 Agent-Based Kinematic Analysis of Structural Mechanics

2.1 Task description

Typical structural example images are selected from structural mechanics textbooks. The agent is required to invoke skill packages to perform kinematic analysis on a given structural image and correctly identify geometrically stable and geometrically unstable systems.

2.2 Hardware and Software Environment

The operating system is Windows 11. The software versions used are Python 3.12.4, LangChain 1.4.2, and DeepAgents 0.7.19. The hardware includes an AMD Ryzen 5 2600 CPU, 64 GB DDR4 memory, and an RTX 4060 Ti 16 GB GPU.

2.3 Skills

The agent invokes two skills: doubao-visualization and structural-kinematic-analysis.

1. doubao-visualization is a visualization analysis skill package provided by the Doubao platform. It has image understanding capability and can generate HTML-rendered visual content.

The agent invokes it to understand the input structural example images and to draw schematic diagrams annotated with kinematic analysis results.

2. structural-kinematic-analysis is a kinematic analysis skill package developed according to structural mechanics textbooks. It can understand nodes, bars, and supports, and can perform kinematic analysis using the binary-body, two-rigid-plate, and three-rigid-plate rules. It consists of files such as SKILL.md, freedom_degree.py, rules.md, and examples.md.

The agent invokes it to calculate the degrees of freedom of the structure and to determine whether the structure is a geometrically stable or geometrically unstable system.

2.4 Construction of the Agent

1.Loading the large language model

ChatOpenAI is a class in the LangChain framework used to interact with large language models.

Here we adopt the doubao-seed-evolving that is deep-thinking model, which was developed by ByteDance for code-development and agent scenarios.

The code is shown in the following figure:

```
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
import os
load_dotenv()
llm = ChatOpenAI(
        model=os.getenv("ARK_MODEL", "doubao-seed-evolving"),
        api_key=os.getenv("ARK_API_KEY"),
        base_url=os.getenv("ARK_BASE_URL", "https://ark.cn-beijing.volces.com/api/v3"),
        temperature=0,    )
```

Fig. 3 Code for loading the large language model

2.Creating the agent, loading skill packages, and setting the system prompt

DeepAgents is a library for building agents, built on top of LangGraph, designed to make it easier for developers to create agents that can plan, decompose tasks autonomously, and execute complex workflows. The two skill packages, doubao-visualization and structural-kinematic-analysis, are placed in advance in the `./skills` directory for the agent to invoke.

InMemorySaver is a checkpoint saver in LangGraph for short-term memory; it stores the conversation state directly in process memory (RAM). Because all memory is lost once the process restarts, it is only suitable for development and testing and cannot be used in production.

The agent's role, behavior, and output format are determined by setting a system prompt.

The code is shown in the following figure:

```
from deepagents import create_deep_agent
from deepagents.backends import LocalShellBackend
from langgraph.checkpoint.memory import InMemorySaver
_py_dir = os.path.dirname(sys.executable)
agent = create_deep_agent(
    model=llm,
    backend=LocalShellBackend(
        root_dir=".",
        virtual_mode=True,
        inherit_env=True,
        env={"PATH": _py_dir + os.pathsep + os.path.join(_py_dir, "Scripts") + os.pathsep + os.environ.get("PATH", "")},
    ),
    skills=["./skills"],
    checkpointer=InMemorySaver(),
    system_prompt=(
"""You are a Structural Mechanics Kinematic Analysis Assistant. When analyzing a structural schematic diagram,
 output in the following order:
Step 1 — Read the diagram and draw an annotated structural schematic: Identify the nodes, members, and
supports, and output an HTML/SVG schematic according to the doubao-visualization specification. Mark
the node numbering, all members, support symbols (a fixed hinge = 2 link bars; a movable hinge = 1 link bar),
and constraint descriptions. The figure must be a directly renderable ```html code block, with a white background.
Step 2 — Perform the kinematic analysis: Confirm the number of nodes j, members b, and support link bars r →
calculate the degree of freedom W (for trusses: W = 2j - b - r; for rigid frames: W = 3m - 2h - r) → verify with
the freedom_degree script → conduct the geometric composition analysis using the binary-body, two-rigid-plate,
and three-rigid-plate rules (pay attention to degenerate cases such as three hinges being collinear and hinges
at infinity) → conclude: geometrically stable (statically determinate / statically indeterminate) / instantaneously unstable
/ permanently variable, together with the number of redundant constraints."""
    ),
)
```

Fig. 4 Code for creating the agent, loading skill packages, and setting the system prompt

3. Inputting the structural example image, setting the user prompt, invoking the agent, and outputting text results

The code is shown in the following figure:

```
from langchain_core.messages import HumanMessage
IMAGE = "./truss-1.png"
#IMAGE = "./truss-2.png"
question = (
f"""Perform a structural mechanics kinematic analysis on the structural schematic {IMAGE}.
Step 1: First, carefully inspect the image and draw an annotated structural schematic following the
doubao-visualization specification (HTML/SVG renderer, in a ```html code block) — mark all node numbers,
all members, and support symbols (fixed hinge / movable hinge and their respective number of link bars).
Step 2: Then perform the kinematic analysis — confirm j/b/r → calculate W (verified by the freedom_degree script) →
carry out the geometric composition analysis using the binary-body / two-rigid-plate / three-rigid-plate rules → give
the conclusion (geometrically invariant / instantaneously unstable / permanently variable, plus the number of
redundant constraints, plus statically determinate / statically indeterminate)."""
)

result = agent.invoke(
    {"messages": [HumanMessage(content=question)]},
    config={"configurable": {"thread_id": "analysis-1"}},
)

for m in result["messages"]:
    if m.type == "ai" and m.content:
        print(m.content)
        print("\n" + "-" * 60)
```

Fig. 5 Code for inputting the structural example image, setting the user prompt, invoking the agent, and outputting text results

4. Rendering the schematic diagram with kinematic analysis results output by the agent

Here the agent generates an HTML/SVG code block, which can be rendered directly in clients that support a renderer; otherwise it degrades to plain code display. The author used the Jupyter Notebook development environment, so the HTML code block needs to be extracted from the agent's reply and displayed in Jupyter Notebook via IPython as an annotated schematic diagram.

The code is shown in the following figure:

```python
def extract_renderer_html(text):
    """Extract the ```html code block content from the agent's reply (for rendering inside the notebook)."""
    import re
    m = re.search(r"```html[^\n]*\n(.*?)```", text, re.S)
    return m.group(1) if m else None

from IPython.display import display, HTML
html_code = None
for m in reversed(result["messages"]):
    if m.type == "ai" and m.content:
        html_code = extract_renderer_html(m.content)
        if html_code:
            break

if html_code:
    display(HTML(html_code))
else:
    print("No HTML code block found. Message diagnostics (last 6):")
    for i, m in enumerate(result["messages"][-6:]):
        c = (str(m.content)[:150] if m.content else "(empty)")
        print(f"  [{i}] type={m.type} | content={c}")
```

Fig. 6 Code for rendering the schematic diagram with kinematic analysis results output by the agent

5. Optional interactive dialogue

Because the agent has memory capability, follow-up questions can be asked consecutively, for example: "Is it still stable if the right support is removed?"

The code is shown in the following figure:

```python
while True:
    q = input("User message (type exit to quit): ")
    if q.lower() == "exit":
        break
    res = agent.invoke(
        {"messages": [HumanMessage(content=q)]},
        config={"configurable": {"thread_id": "chat-1"}},
    )
    for m in res["messages"]:
        if m.type == "ai" and m.content:
            print(m.content)
            print("\n" + "-" * 60)
```

Fig. 7 Code for optional interactive dialogue

2.5 Agent Testing

1. Test 1

A structural example image of a geometrically stable system is input, as shown in the following figure:

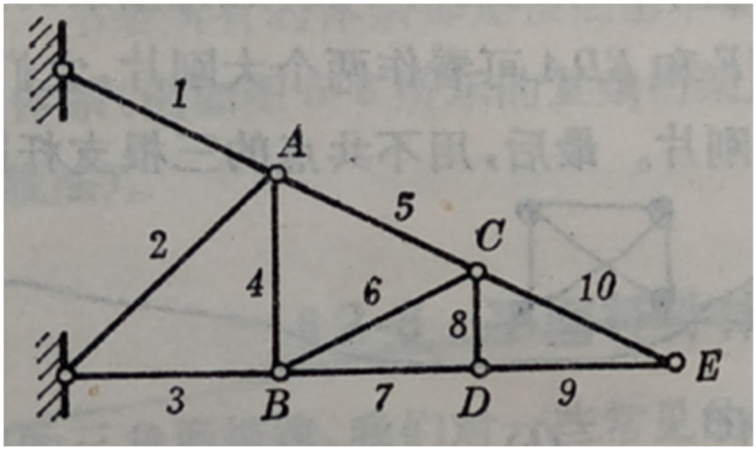


Fig.8 Example of geometrically stable system inputting to the agent

The schematic diagram output by the agent is shown in the following figure:

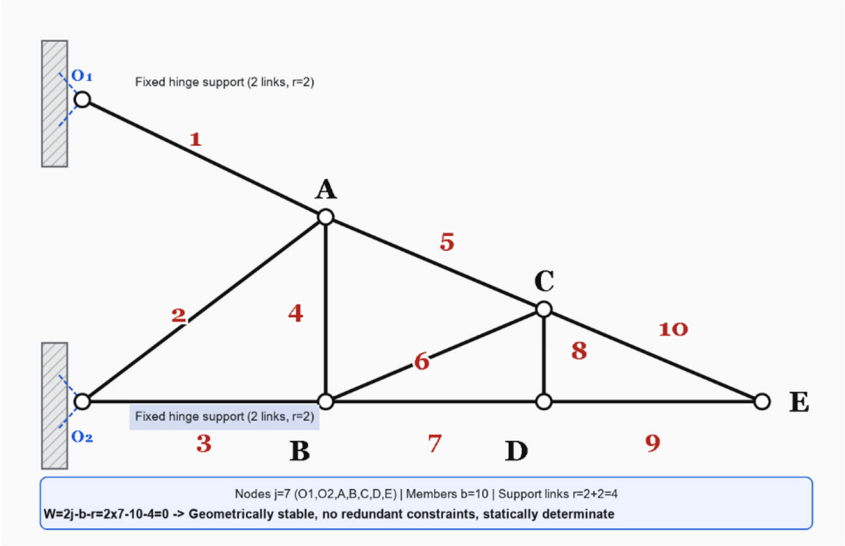


Fig.9 Agent analysis results of example of geometrically stable system

The agent's output is correct.

2.Test 2

A structural example image of a geometrically unstable system is input, as shown in the following figure:

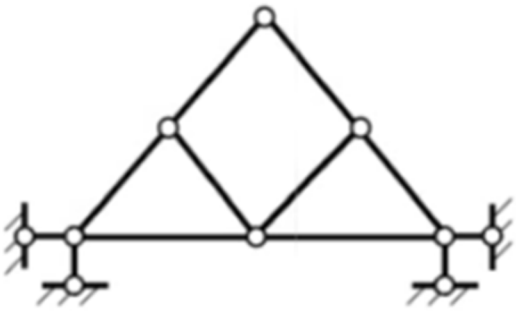


Fig.10 Example of geometrically unstable system inputting to the agent

The schematic diagram output by the agent is shown in the following figure:

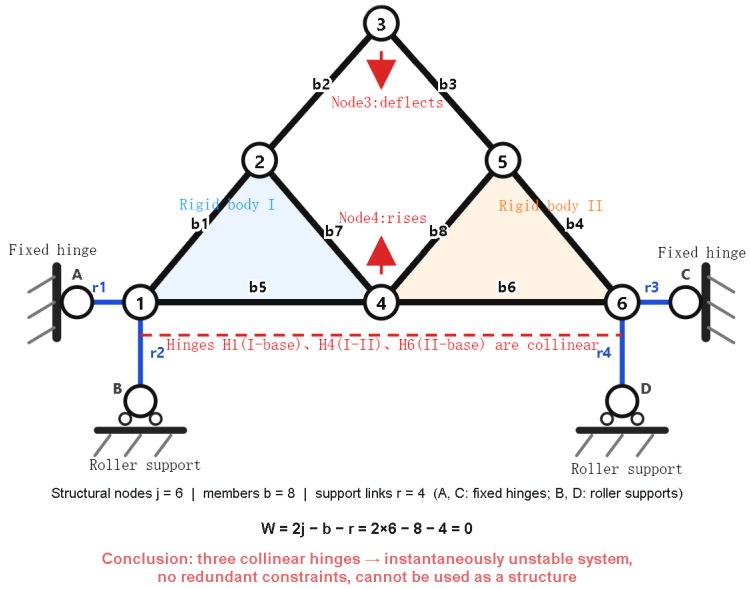


Fig. 11 Agent analysis results of example of geometrically unstable system

The agent's output is correct.

3 Conclusion

This paper targets the kinematic analysis of planar bar systems — a foundational problem in structural mechanics—and explores the feasibility of introducing AI agents into engineering analysis. Based on the LangChain framework, and taking a large language model as the reasoning core, we integrate the image-understanding and visualization skill doubao-visualization together with the textbook-based professional skill package structural-kinematic-analysis to build an agent that can autonomously perceive structural images, compute the degrees of freedom of the system, and invoke rules such as the binary-body, two-rigid-plate, and three-rigid-plate rules to perform kinematic analysis. Tests on two typical examples—a geometrically stable system and a geometrically unstable system—show that the agent produces correct verdicts in both cases and outputs schematic diagrams annotated with analysis results. Thanks to its memory capability, the system also supports consecutive follow-up questions such as "Is it still stable after a certain support is removed?", thereby demonstrating an initial ability to assist structural mechanics teaching and the preliminary assessment of engineering structures.

4 Discussion

The author believes that the entrepreneurship track of agents—which may be regarded as plug-ins to large models—has no future. The reason is that general-purpose foundation large models have already reached the AGI singularity in the digital world (the next step being continuous evolution and entry into the physical world), and the speed at which entrepreneurs spend time and effort developing and refining agents is slower than the speed at which large models internalize agents, turning them into neural-network weights. Agents are therefore merely transitional technology, and large models will ultimately swallow every such plug-in on the market. In other words, large models will

weight all intelligence and, just like humans, ultimately reduce everything to neuronal parameters.

References

[1] Yuqiu LONG, Shihua BAO. Structural Mechanics Course (the first volume) [M]. Beijing: Higher Education Press, 1988.

[2] Liankun LI. Structural Mechanics (the first volume) [M]. 4th ed. Beijing: Higher Education Press, 2004.

[3] Lingyan ZHANG, Peiwei NI, Qin LIU, et al. Preliminary study on construction methods of intelligent agent for famous doctors diagnosis and treatment of Alzheimer's disease [J]. Chinese Traditional and Herbal Drugs, 2026(14).

[4] Hanyue MAO, Mingwen TONG, Jia CHEN, et al. Multi-Agent Empowered AI-Generated Digital Educational Resources Development: Model Construction and Implementation Safeguards [J]. Modern Educational Technology ,2026(7): 130–138.

[5] Rongrong CHEN, Ruonan SHI. Construction and application of a novelty-search agent based on human-machine collaboration [J]. Research On Library Science ,2026(6): 73–80.

[6] Fan MA, Yushun LI, Rui JIANG, et al. Educational Assessment Agents for Multidimensional Scenarios: Development,Application, and Effectiveness Validation [J]. Modern Educational Technology ,2026(7): 120–129.

[7] Yunqiang YANG, Hongyan LI, Jin WU. Agent Construction and Application Development (Python + LangChain) [M]. Beijing: Posts and Telecom Press, 2026.